

Data Structure

04. 스택

4주차: 스택 & 재귀 호출 스택

? 오늘의 핵심 질문

? 질문 1

재귀 함수는 내부적으로 어떻게 실행될까?

? 질문 2

함수가 끝나면 어떻게 이전 함수의 실행 위치로 돌아갈 수 있을까?

? 질문 3

괄호 검사를 어떻게 스마트하게 해결할 수 있을까?

목차 INDEX

01 스택

- 스택
- LIFO & FIFO
- Push
- Pop
- 시간복잡도

02 스택의 활용

- Undo & Redo
- 괄호 검사

03 Call Stack

- 함수 호출 & 콜 스택
- Stack Frame
- 재귀 함수의 동작
- Stack Overflow
- 재귀 & 반복문

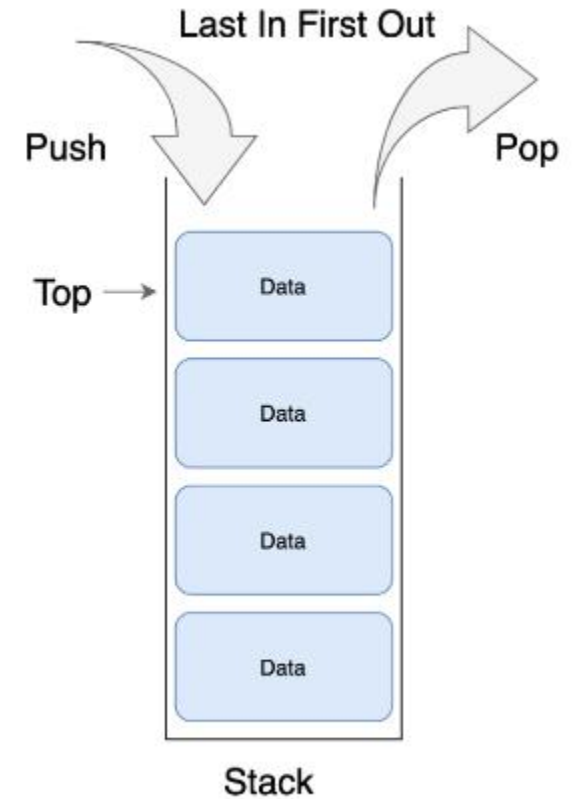
01

스택

Stack

이번 시간에 볼 거: 데이터를 어떤 순서로 꺼낼 것인가?

- 지금까지는 연결 리스트, ADT(추상 자료형) 등을 학습하며 데이터를 어떻게 저장할지를 주로 다뤘음
- 이번에는 그렇게 저장한 데이터를 어떻게 꺼낼지에 대해 공부해 볼거임



이번 시간에 볼 거: 데이터를 어떤 순서로 꺼낼 것인가?

02 추상 자료형 Abstract Data Types

추상 자료형의 예시: Stack

↑ **push(item)**

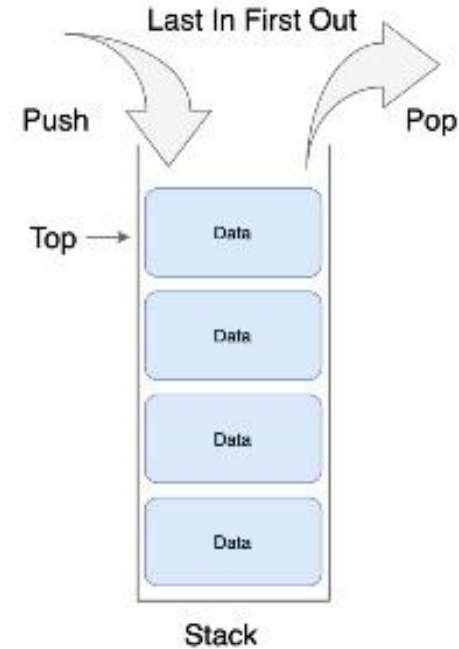
스택 맨 위에 새로운 데이터 추가

↓ **pop()**

맨 위에 있는 데이터 제거 후 반환

🔍 **peek() / is_empty()**

맨 위 확인 / 비었는지 검사



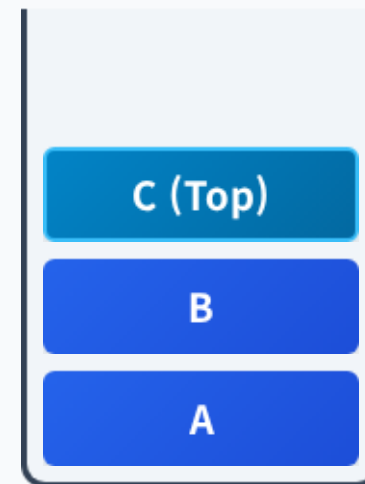
Stack?

정의

한쪽 끝(Top)에서만 데이터의 **삽입(Push)**과 **삭제(Pop)**가 이루어지는 제한된 구조의 선형 자료구조

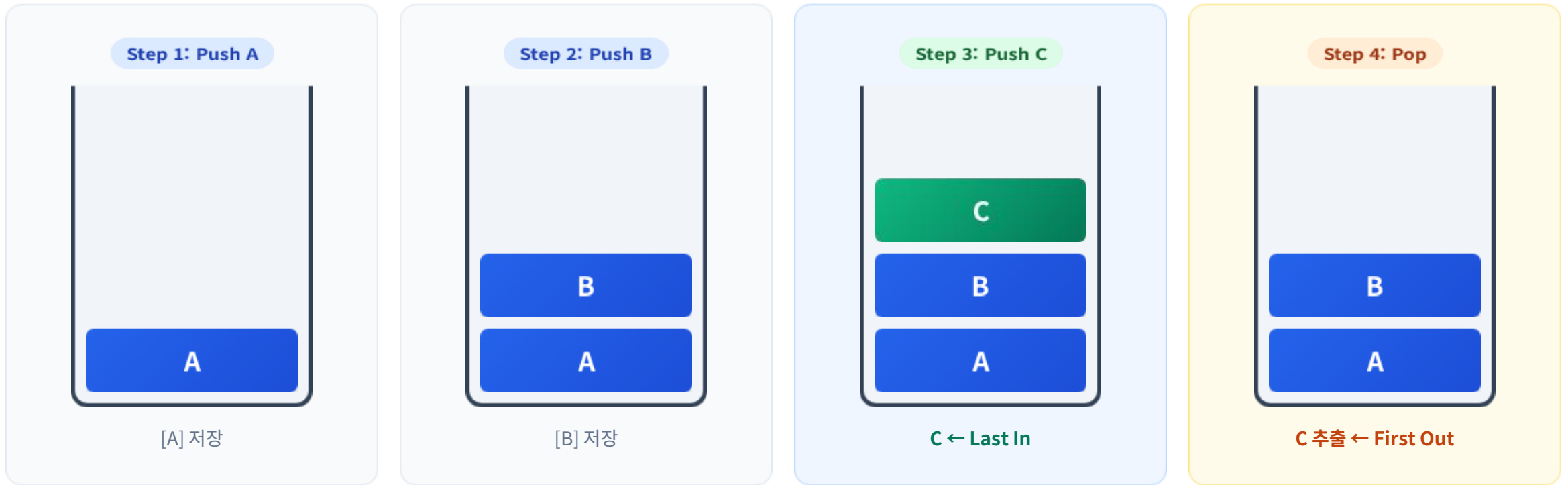
- **LIFO (Last-In, First-Out):** 후입선출 - 가장 나중에 들어온 것이 가장 먼저 나감
- **입출력은 한 곳에서:** 출입구가 하나뿐인 막힌 원통 형태
- **Top (Peak):** 데이터가 들어오고 나가는 유일한 최상단 지점
(TOP에 있는 데이터의 주소를 가리키는 포인터가 있고, 주로 Top Pointer라 부름)

TOP ↓



입력 순서: A → B → C
출력 순서: C → B → A

LIFO: Last In, First Out



스택의 연산

push(item)

삽입

스택의 가장 맨 위(Top)에 새로운 데이터를 추가

pop()

삭제 & 반환

스택의 가장 맨 위(Top)에 있는 데이터를 제거하고 반환

peek() / top()

조회

가장 맨 위(Top) 데이터를 제거하지 않고 값만 확인

상태 확인 is_empty()

스택이 비어있는지 여부(True/False)를 확인

상태 확인 size()

스택에 현재 저장된 요소의 총 개수를 반환

Stack은 “추상 자료형(ADT)” 임

리스트(Python)로 만들든 연결 리스트(Linked List)로 만들든, 이 연산 규칙을 따르면 모두 스택임

스택의 Push 연산의 원리

push(data) 단계별 과정

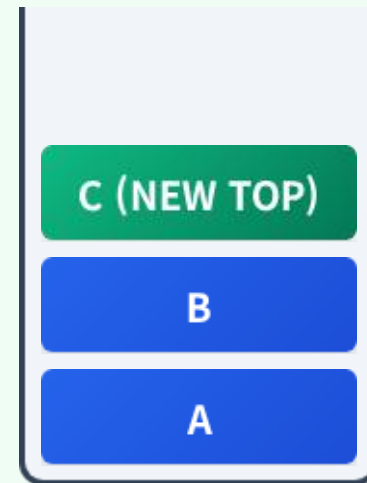
1. 현재 스택의 최상단 지점(Top, 이전 시점에서 B)을 확인한다.
2. Top 바로 위 위치에 새로운 데이터를 배치한다.
3. Top 포인터의 위치를 새 데이터를 가리키도록 1 증가시킨다.

Push 시간복잡도

$O(1)$ - 상수 시간

기존 데이터 위치 변경 없이 맨 끝에 하나만 추가하므로 매우 빠름

push('C') 실행 시 시각화



스택의 Pop 연산의 원리

pop() 단계별 과정

1. 스택이 비어있는지(isEmpty) 먼저 확인
2. Top 위치의 데이터를 추출하여 임시 변수에 저장
3. Top 포인터 위치를 아래로 1 감소시키고 데이터를 반환

Pop 시간복잡도: $O(1)$

⚠ Stack Underflow 오류

비어 있는 스택(Empty Stack)에서 데이터를 꺼내려고(pop) 시도할 때 발생하는 예외 상황임

```
IndexError: pop from empty list
```

스택의 주요 연산

연산 (Operation)	시간 복잡도 (Time Complexity)	설명
Push (삽입)	$O(1)$ (평균)	맨 끝에 원소 추가
Pop (삭제 및 반환)	$O(1)$	맨 끝 원소 제거
Peek / Top (조회)	$O(1)$	맨 끝 원소 읽기
isEmpty / Size	$O(1)$	스택의 전체 길이 정보 조회
Search (탐색)	$O(N)$	특정 값이 스택 내부 어디에 있는지 찾으려면 전체 순회 필요

스택은 빠른 검색을 위한 자료구조는 아니고, 데이터 처리 순서를 제한(LIFO)하기 위한 자료구조임

02

스택의 활용

Stack 101: How to use it

스택의 주요 쓰임새



괄호 검사

컴파일러의 수식 parsing 및 괄호 짝 맞
추기



Undo / Redo

문서 편집기 등의 작업 실행 취소 기능



브라우저 방문기록

웹 브라우저의 뒤로 가기 / 앞으로 가기



함수 호출 (Call Stack)

프로그램 실행 중 함수 서브루틴 관리

스택은 주로 괄호가 짝이 맞는지, 되돌리기 기능,
방문 기록 등에 쓰임

Call Stack에도 쓰이는데, 뒤에서 알아볼 예정

나중에 스택을 응용해서 DFS 등을 할 거라 확실히 알아둬야 함

Undo / Redo

사용자의 문서 편집 작업 흐름

1. 텍스트 입력 ("Hello")
2. 굵게 적용 (Bold)
3. 이미지 삽입
4. 글자색 변경 (Red) ← 최근 작업

Undo (Ctrl + Z) 실행 시

첫 번째 Undo:
"글자색 변경 취소" (가장 최근 작업 Pop!)

두 번째 Undo:
"이미지 삽입 취소" Pop!

실제 상용 프로그램에서는 Undo Stack,
Redo Stack 두 개를 써서 더 정교하게 관리함

괄호 짝 검사

올바른 괄호 문자열 (Valid)

((()))

모든 여는 괄호가 올바른 순서로 대응하는 닫는 괄호를 만남

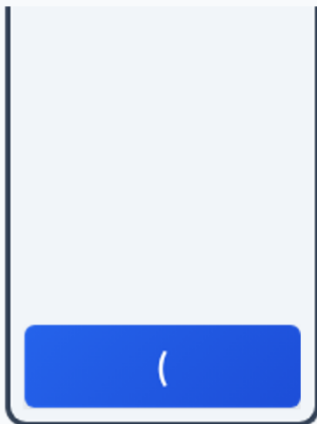
잘못된 괄호 문자열 (Invalid)

((())))

닫는 괄호가 더 많거나 짝의 종류가 일치하지 않음

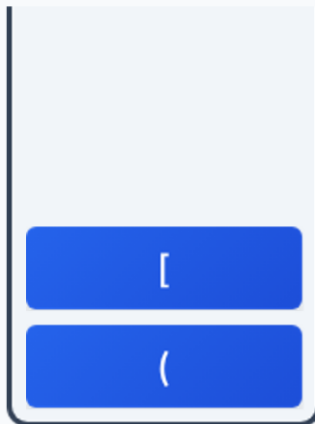
1. '(' 등장

Push '('



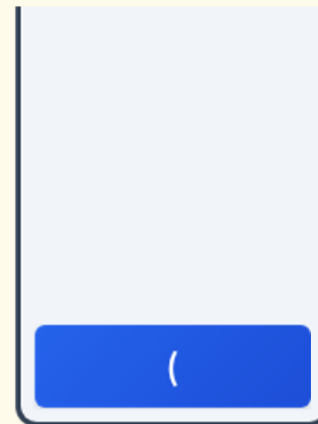
2. '[' 등장

Push '['



3. ']' 등장

Pop 후 '['와 짝 검사



4. ')' 등장

Pop 후 '('와 짝 검사



스택 비어있음 (성공)

03

호출 스택

Call Stack

함수 호출 시 문제

```
def b():  
    print('B')  
  
def a():  
    print("A")  
  
b()  
a()  
print("출력 끝")
```

여기서 b() 실행하면 위의 def 부분을 실행하게 됨
그런 다음 어떻게 a() 줄로 되돌아올 수 있는거임?

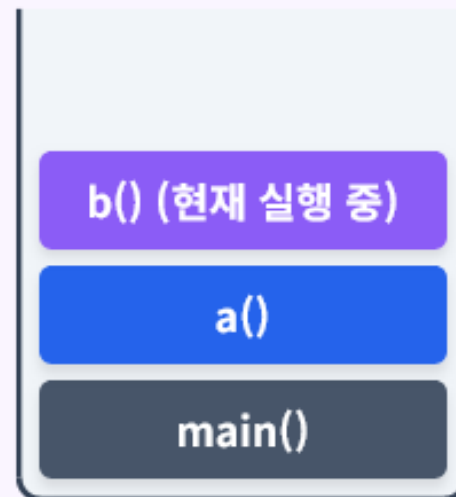
함수 호출 스택 (Call Stack)

Call Stack 정의

프로그램이 함수 호출과 반환(Return) 상태를 관리하기 위해 사용하는 메모리 상의 Stack 구조 실행 영역

-  함수 호출 시: 실행 정보를 스택에 Push
-  함수 종료(Return) 시: 최상단 실행 정보를 Pop
-  LIFO 연결: 가장 최근 호출된 함수가 가장 먼저 종료됨!

Call Stack 상태 변화



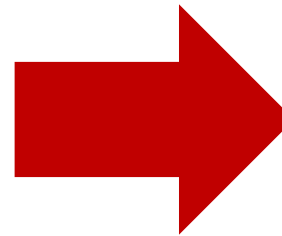
Stack Frame

- 함수가 호출될 때 하나의 Stack Frame이 Call Stack에 쌓임
- 이후 함수의 실행이 종료되면 해당 Frame은 제거됨
- Stack Frame 안에는 Parameter, Variables, Return Address 등이 포함됨

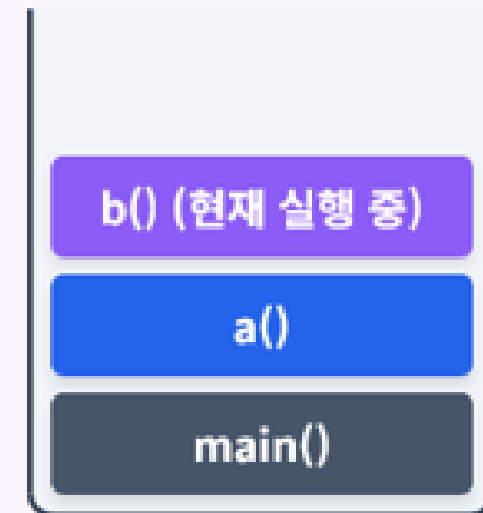
Stack Frame 내부 구성 정보

- 매개변수 (Parameters): 함수에 전달된 인자 값
- 지역 변수 (Local Variables): 함수 내부에서 선언된 변수
- 복귀 주소 (Return Address): 함수가 끝난 후 되돌아갈 코드 위치

이거 하나하나가 모두 Stack Frame임



Call Stack 상태 변화



함수 실행 시 흐름

```
def multiply(a, b):  
    result = a * b  
    return result  
  
def calculate():  
    x = multiply(3, 4)    # multiply 호출  
    return x + 1  
  
answer = calculate()    # 메인 실행  
print(answer)
```

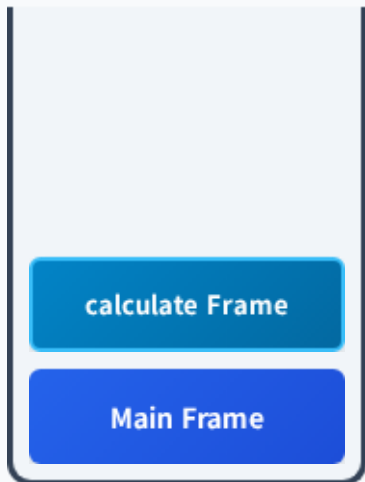
실행 흐름

1. calculate() 호출
2. multiply(3, 4) 호출
3. multiply 연산 완료 후 12 반환
4. calculate로 복귀하여 $12 + 1 = 13$ 연산
5. 최종 값 13 출력

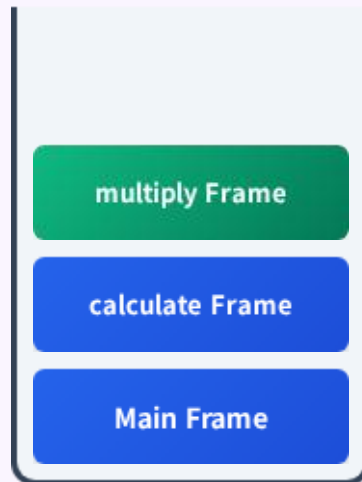
함수 실행 시 흐름: Call Stack

```
def multiply(a, b):  
    result = a * b  
    return result  
  
def calculate():  
    x = multiply(3, 4)    # multiply 호출  
    return x + 1  
  
answer = calculate()    # 메인 실행  
print(answer)
```

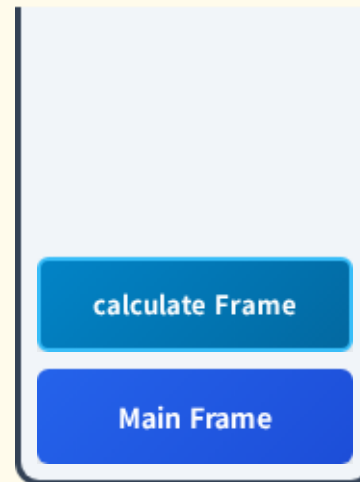
1. calculate() 호출



2. multiply() 호출

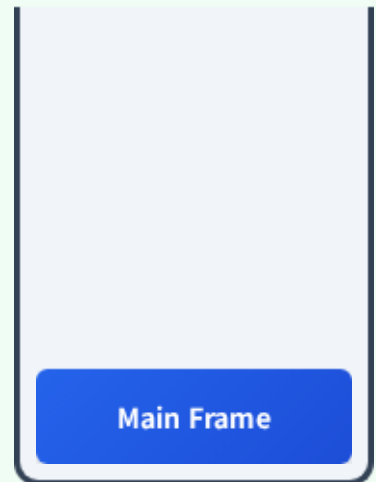


3. multiply() 리턴(Pop)



12 반환 후 Frame 제거

4. calculate() 리턴



13 반환 후 완결

재귀 함수의 동작

```
def factorial(n):  
    # 1. Base Case (종료 조건)  
    if n == 1:  
        return 1  
  
    # 2. Recursive Case (재귀 단계)  
    return n * factorial(n - 1)
```

간단한 팩토리얼 예제를 볼거임

재귀 함수의 조건

1. Base Case (종료 조건)

재귀 호출을 멈추고 값을 반환하는 탈출 조건임
없으면 무한 루프함

```
if n == 1:  
    return 1 # Base Case
```

2. Recursive Case (재귀 단계)

문제를 더 작은 단위로 분할하여 자기 자신을 호출하는 부분

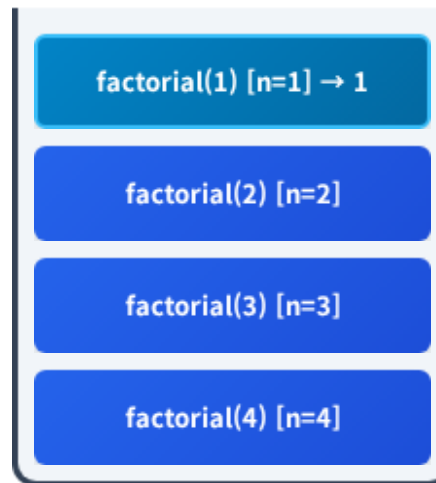
```
return n * factorial(n - 1)
```


재귀 함수의 흐름

factorial(4) 호출 흐름

```
factorial(4) = 4 * factorial(3)
└─ factorial(3) = 3 * factorial(2)
  └─ factorial(2) = 2 * factorial(1)
    └─ factorial(1) = 1 Base Case!
```

Call Stack 반환 과정



Stack Overflow

🔥 Call Stack 메모리 고갈

Call Stack에 프레임이 계속 쌓이다가
지정된 메모리 용량을 초과하는 현상

주요 원인

- ❌ Base Case를 작성하지 않았거나 조건이 잘못된 경우
- ❌ 재귀 호출의 깊이(Depth)가 너무 깊은 경우
(재귀를 너무 많이 반복했을 경우)

```
# Base Case가 없는 무한 재귀 예시
def infinite_recursion(n):
    print(n)
    return infinite_recursion(n + 1)

infinite_recursion(1)
```

```
Traceback (most recent call last):
  File "Infinite_Recursion.py", line 6, in <module>
    infinite_recursion(1)
  File "Infinite_Recursion.py", line 4, in infinite_recursion
    return infinite_recursion(n + 1)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "Infinite_Recursion.py", line 4, in infinite_recursion
    return infinite_recursion(n + 1)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "Infinite_Recursion.py", line 4, in infinite_recursion
    return infinite_recursion(n + 1)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Stack Overflow



stack overflow

코딩판의 지식인같은 사이트인데,
여기의 Stack Overflow가 이런 뜻임

로고를 보면 스택에 Stack Frame이
넘쳐 흐르는(Overflow) 것을 볼 수 있음

재귀와 반복문의 차이

비교 항목	재귀 함수 (Recursion)	반복문 (Iteration)
구현 방식	자기 자신을 다시 호출	for, while 구조 반복 실행
메모리 공간	O(N) Call Stack 프레임 누적	O(1) 추가 메모리 거의 없음
속도 / 성능	함수 호출 오버헤드로 다소 느림	오버헤드가 없어 상대적으로 빠름
가독성 / 장점	트리/그래프, 분할정복 시 코드가 매우 간결함	직관적이며 Stack Overflow 위험이 없음